Computing - Exercise 2

Group 1

Exercise 25: Logstar

The function logstar gives the smallest n such that the n-th iterate of the natural logarithm results in a value less than one. For example, $logstar(123) = 3$, because the second iterate $\log(\log(123)) = 1.571151 \geq 1$ and the third iterate $\log(\log(\log(123))) = 0.4518085 < 1$. Implement logstar.

```
logstar <- function(n) {
  x <- 0
  if (n<=0) {
    return ("log is only defined for positive numbers")
  }
  if (n<1) {
    return (0)
  }
  else {
    while (n>=1) {
      n <- log(n)
      x <- x+1
    }
    return(x)
  }
}

logstar (75)
```

```
## [1] 3
```

```
logstar(123)
```

```
## [1] 3
```

```
logstar(exp(100))
```

```
## [1] 4
```

```
logstar(0.5)
```

```
## [1] 0
```

Exercise 26 – Greatest Common Divisor (GCD)

The greatest common divisor (GCD) gcd(a, b) can be obtained using Euclid's algorithm which is based on the following two observations:

(a) If b is a divisor of a, then $gcd(a, b) = b$.
(b) If r is the remainder in the integer division of a by b, then $gcd(a, b) = gcd(b, r)$.

Write a function gcd which computes the GCD using Euclid's algorithm. See also http://www.cut-the-knot.org/blue/Euclid.shtml.

```
# gcd = largest positive integer that divides both numbers

gcd <- function(a, b) {
  # signs don't change the gcd, so we just use absolute values
  a <- abs(a)
  b <- abs(b)
  if (a == 0 && b == 0) { # gcd(0, 0) is not defined
    return(NA_integer_)
  }
  if (b == 0) { # gcd(a, 0) = a, and we can't divide by 0 in the loop
    return(a)
  }
  while (a %% b != 0) {
    r <- a %% b
    a <- b
    b <- r
  }
  return(b)
}

# example: (a,b) = (45, 35)
gcd (45,35)
```

```
## [1] 5
```

```
gcd(20,-85)
```

```
## [1] 5
```

```
gcd(20,0) # for 0 every number divides it, so gcd(a,0) = a
```

```
## [1] 20
```

Exercise 27 – Evenness Pattern

Write a function which, for a given natural number n, returns a logical vector containing for each $i \leq n$ in reverse order, a sequence of length i indicating whether i is even or not. E.g., for $n = 3$ the function should return c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE).

```r
even_or_not <- function(n) {

  result <- c()

  for (i in n:1) {
    i_vec <- rep(i %% 2 == 0, times = i)
    result <- c(result, i_vec)
  }
  return(result)
}

even_or_not(3)  # should return the example output
```

```
## [1] FALSE FALSE FALSE  TRUE  TRUE FALSE
```

```r
even_or_not(4)  # should return 4 times TRUE and then ever_or_not(3)
```

```
##  [1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE  TRUE FALSE
```

Exercise 28 – Counting Odd Numbers

Write a function which computes the number of odd numbers in a given sequence of natural numbers. (Hint: use sum to count the number of TRUEs in a logical vector.)

```r
count_odd <- function(n){
  paste0("There are ", sum(n %% 2 == 1), " odd numbers in the given sequence")
  }

count_odd(c(1,4,3,5,7,65,32))
```

```
## [1] "There are 5 odd numbers in the given sequence"
```

Exercise 29 – Relatively Prime Numbers

Write a function which returns (a logical indicating) whether two given natural numbers are relatively prime (i.e., have GCD 1).

```r
# uses gcd from exercise 26, relatively prime means gcd = 1
relatively_prime <- function(a, b) {
  return(gcd(a,b) == 1)
}

# examples: (a,b) = (7,3), (45,35)
relatively_prime(7,3)
```

```
## [1] TRUE
```

```
relatively_prime(45,35)
```

```
## [1] FALSE
```

Exercise 30 – Prime Number Test

Write a function is_prime returning (a logical indicating) whether a given natural number is a prime number or not.

```
is_prime <- function(n) {
  if (n <= 1 || n %% 1 != 0) { # only natural numbers > 1 can be prime
    return(FALSE)
  }
  if (n == 2) {
    return(TRUE)
  }
  else if(any(n %% 2:(n-1) == 0)) {
    return(FALSE)
  }
  else {return(TRUE)}
}

# examples: n = 27, 29
is_prime(27)
```

```
## [1] FALSE
```

```
is_prime(29)
```

```
## [1] TRUE
```

```
is_prime(1)
```

```
## [1] FALSE
```

```
is_prime(2)
```

```
## [1] TRUE
```

```
is_prime(0)
```

```
## [1] FALSE
```

Exercise 32 – Perfect Numbers

A perfect number is a positive integer which equals the sum of its proper divisors. The smallest perfect number is $6 = 1 + 2 + 3$. Write a function which tests whether a given positive integer is a perfect number, and determine the first 4 perfect numbers via simple iteration. (See http://en.wikipedia.org/wiki/Perfect_number for more information.)

```r
is_perfect <- function(n) {
  if (n == 1) {
    return(FALSE) # 1 is not perfect by definition
  }
  # all numbers from 1 to n-1 that divide n (proper divisors)
  divisors <- (1:(n-1))[n %% (1:(n-1)) == 0]
  return(sum(divisors) == n)
}

# examples: n = 6, 12
is_perfect(6)
```

```
## [1] TRUE
```

```r
is_perfect(12)
```

```
## [1] FALSE
```

```r
# first 4 perfect numbers: just try n = 2, 3, 4, ... until we have 4
first_4_perfect <- c()
n <- 2

while(length(first_4_perfect) < 4) {
  if(is_perfect(n)) {
    first_4_perfect <- c(first_4_perfect, n)
  }
  n <- n + 1
}
first_4_perfect
```

```
## [1]    6   28  496 8128
```

Exercise 33 – Highly Composite Numbers

A highly composite number is a positive integer that has more divisors (i.e. positive integers that divide into this number) than any smaller positive integer has (https://en.wikipedia.org/wiki/Highly_composite_number). Write a function which returns as a logical value if a given number n is highly composite or not and find the first 20 highly composite numbers.

```r
is_highly_composite <- function(n) {
  # Highly composite numbers are positive integers (>= 1)
  # Anything less than 1 prevents valid sequence generation
  # and avoids modulo by 0 crashes
  if (n < 1) return(FALSE)

  # 1 is considered highly composite by definition (no smaller positive integers exist)
  if (n == 1) return(TRUE)

  # Calculate divisors for the target number
  divisors_n <- sum(n %% 1:n == 0)
```

```r
  # Loop through all smaller numbers to check if any have the same or more divisors
  for (k in 1:(n - 1)) {
    divisors_k <- sum(k %% 1:k == 0)

    if (divisors_k >= divisors_n) {
      return(FALSE) # if we find such a number, break directly
    }
  }

  return(TRUE)
}

is_highly_composite(200)
```

```
## [1] FALSE
```

```r
hcn_vector <- numeric(0)  # Empty vector to store the results
current_num <- 1          # Start checking from 1
max_divisors_seen <- 0    # Tracker for the current "record"

while (length(hcn_vector) < 20) {
  # Count divisors for the current number
  current_divisors <- sum(current_num %% 1:current_num == 0)

  # If it sets a new record, it's a highly composite number
  if (current_divisors > max_divisors_seen) {
    hcn_vector <- c(hcn_vector, current_num)
    max_divisors_seen <- current_divisors # Update the record
  }

  current_num <- current_num + 1
}

# Output the result
hcn_vector
```

```
##  [1]    1    2    4    6   12   24   36   48   60  120  180  240  360  720  840
## [16] 1260 1680 2520 5040 7560
```

Exercise 34 – Collatz Sequence

Consider the following sequence: start with any positive integer n_0, and, subsequently, obtain every term n_i for $i \geq 1$ from the previous one as follows:

$$n_i = \begin{cases} \dfrac{n_{i-1}}{2}, & \text{if } n_{i-1} \text{ is even} \\ 3n_{i-1} + 1, & \text{otherwise.} \end{cases}$$

According to the (still unproven) Collatz conjecture (https://en.wikipedia.org/wiki/Collatz_conjecture), this sequence will always reach 1, no matter what initial value of n_0. Write a function to verify the Collatz conjecture for $1 \leq n_0 \leq 100,000$. Which starting value n_0 in this range requires the greatest number of steps to reach 1? How many steps does it need?

```r
# returns the value the sequence ends at and how many steps it took

collatz <- function(n) {

  if (n < 1) {
    warning("n must be >= 1")
    return(NA)
  }

  steps  <- 0

  while(n != 1) {
    if(n %% 2 == 0) { # n is even
      n <- n / 2
    } else {
      n <- 3*n + 1
    }
    steps <- steps + 1
  }

  return(steps)
}

# the function should output the sequence, not the steps

collatz_test    <- 1:1e5
max_steps       <- 0
which_max_steps <- 0

for(i in collatz_test) {
  steps <- collatz(i)

  if(steps > max_steps) {
    max_steps       <- steps
    which_max_steps <- i
  }
}

c(n_0 = which_max_steps, Steps = max_steps)
```

```
##   n_0 Steps
## 77031   350
```

Every starting value reaches 1, so the conjecture holds for $1 \leq n_0 \leq 100{,}000$. The starting value $n_0 = 77031$ needs the most steps, namely 350, so its sequence has 351 terms.

Exercise 35 – Binary Representation

Write a function which computes the binary representation (as a sequence of 0s and 1s) of a given natural number (e.g., for x = 6 the function should return c(1, 1, 0)). Also implement the inverse function.

We repeatedly take the remainder mod 2 (the last binary digit) and then divide by 2 with `%/%`. Example for $n = 13$:

step	n	$r = n \bmod 2$	binary_help	new n
1	13	1	(1)	6
2	6	0	(0, 1)	3
3	3	1	(1, 0, 1)	1
4	1	1	(1, 1, 0, 1)	0

```
binary <- function(n) {
  if (n < 0 || n %% 1 != 0) {
    stop("Please insert a non-negative integer")
  }
  if (n == 0) {
    return(c(0))
  }
  binary_help <- c()
  while (n > 0) {
    r <- n %% 2 # last binary digit (0 or 1)
    binary_help <- c(r, binary_help) # put it in front
    n <- n %/% 2 # remove the last digit
  }
  return(binary_help)
}

# example: n = 10
binary(10)
```

```
## [1] 1 0 1 0
```

```
binary(7)
```

```
## [1] 1 1 1
```

```
binary(15)
```

```
## [1] 1 1 1 1
```

For the inverse, a vector $x = (x_1, \ldots, x_L)$ of 0s and 1s stands for

$$n = \sum_{j=1}^{L} x_j \, 2^{L-j},$$

so the first entry gets the highest power of 2 and the last one gets 2^0. This can be computed in one line without a loop.

```
inv_binary <- function(x) {
  # check every entry, not just the first one
  if (any(x != 0 & x != 1)) {
    return("Only 0s and 1s are allowed. Please insert a binary number")
  }
  n <- sum(2^((length(x)-1):0) * x)
```

```
  return(n)
}

# examples: x = (1, 0, 1, 0), back and forth with binary(), wrong input
inv_binary(c(1,0,1,0))
```

```
## [1] 10
```

```
inv_binary(binary(13))
```

```
## [1] 13
```

```
inv_binary(c(1,2))
```

```
## [1] "Only 0s and 1s are allowed. Please insert a binary number"
```

Exercise 36 – Least Common Multiple (LCM)

Write a function lcm which computes the least common multiple (LCM) of two given natural numbers. Hints: derive a relation between the LCM, the GCD and the numbers, and try avoiding integer overflows when computing the LCM.

We use that

$$\text{lcm}(a,b) \cdot \gcd(a,b) = a \cdot b \quad \Rightarrow \quad \text{lcm}(a,b) = \frac{a}{\gcd(a,b)} \cdot b.$$

Dividing by $\gcd(a,b)$ first means we never compute the (possibly huge) product $a \cdot b$. The `gcd` function is from exercise 26.

```
lcm <- function(a,b) {
  if (a<1 || b<1 || a%%1!=0 || b%%1 != 0) {
    return("Please insert two natural numbers")
  }
  gcd_help <- gcd(a,b)
  return((a/gcd_help) * b) # divide first so a*b doesn't get too big
}

# example: (a,b) = (3,5)
lcm(3,5)
```

```
## [1] 15
```

```
lcm(20,8)
```

```
## [1] 40
```

Exercise 38 – Pascal's Triangle

Pascal's triangle is a triangular array of the binomial coefficients. Implement a function which returns the k-th row of the triangle starting at 0 (so that for example for $k = 4$ we get c(1, 4, 6, 4, 1)).

Row k consists of the binomial coefficients

$$\binom{k}{i} = \frac{k!}{i!\,(k-i)!}, \qquad i = 0, \ldots, k,$$

so it has $k + 1$ entries. Since R starts counting at 1, $\binom{k}{i}$ is stored at position $i + 1$.

```
pascal_row <- function(k) {
  if (k < 0 || k %% 1 != 0) return("k has to be a non-negative Integer")

  row <- integer(k + 1)

  for (i in 0:k) {
    row[i + 1] <- factorial(k) / (factorial(i) * factorial(k - i))
  }
  return(row)
}


# examples
pascal_row(4)
```

```
## [1] 1 4 6 4 1
```

```
pascal_row(5)
```

```
## [1]  1  5 10 10  5  1
```

Exercise 39 – Binomial Model for Stock Prices

The Binomial model is a popular and easy model for stock prices. The starting value of an asset S_0 changes in each time step either by a factor U (up) or by a factor D (down). At timestep N we therefore have $N + 1$ different possibilites $S_{N,i} = S_0 \cdot U^N \cdot D^{N-i}$, where $i = 0, ..., N$. Write a function that returns the values of the asset at time n as list, i.e. a list S, where the n-th element contains the elements $S_{n,i}$ for all $i = 0, ..., n$, (a) using only "for" loops, (b) using the apply function family (i.e., apply, lapply, sapply,. . .). (c) What are the differences of the above methods, which one works better?

At time k with i up moves (and $k - i$ down moves) the value is $S_{k,i} = S_0\, U^i D^{k-i}$, for $i = 0, \ldots, k$. The list element `S[[k+1]]` holds all values at time k (`S[[1]]` is time 0).

```
# (a) using only "for" loops
value_for <- function(S0, n, U, D) {
  S <- list() # S[[k+1]] will hold all values at time k

  for (k in 0:n) { # time steps
    S_k <- c()

    for (i in 0:k) { # i = number of up moves, k+1 possibilities
```

```
      S_k <- c(S_k, S0 * U^i * D^(k - i))
    }

    S[[k + 1]] <- S_k # position k+1 because S[[1]] is time 0
  }
  return(S)
}

# example: (S0, n, U, D) = (10, 5, 1.08, 0.8)
value_for(10, 5, 1.08, 0.8)
```

```
## [[1]]
## [1] 10
##
## [[2]]
## [1]  8.0 10.8
##
## [[3]]
## [1]  6.400  8.640 11.664
##
## [[4]]
## [1]  5.12000  6.91200  9.33120 12.59712
##
## [[5]]
## [1]  4.09600  5.52960  7.46496 10.07770 13.60489
##
## [[6]]
## [1]  3.276800  4.423680  5.971968  8.062157 10.883912 14.693281
```

```
# (b) using the apply functions
value_apply <- function(S0, n, U, D) {
  # lapply goes through the time steps and returns a list,
  # sapply computes all values for one time step as a vector
  S <- lapply(0:n, function(k) {
    sapply(0:k, function(i) S0 * U^i * D^(k - i))
  })
  return(S)
}

# example: (S0, n, U, D) = (10, 5, 1.08, 0.8)
value_apply(10, 5, 1.08, 0.8)
```

```
## [[1]]
## [1] 10
##
## [[2]]
## [1]  8.0 10.8
##
## [[3]]
## [1]  6.400  8.640 11.664
##
## [[4]]
```

```
## [1]  5.12000  6.91200  9.33120 12.59712
##
## [[5]]
## [1]  4.09600  5.52960  7.46496 10.07770 13.60489
##
## [[6]]
## [1]  3.276800  4.423680  5.971968  8.062157 10.883912 14.693281
```

(c) Both give the same result. The for loop version is easier to follow step by step, the apply version is shorter and more typical for R. It also doesn't grow vectors inside a double loop, which gets slow for large n, so the apply version works better.

Exercise 40 – Double Sum

Have a look at the following double sum:

$$\sum_{r=1}^{n}\sum_{c=1}^{r}\frac{c^2}{5+10r^3}$$

Write a function which takes $n \in N$ as input and outputs the above sum (a) using "for" loops (b) using the functions $col()$ and $row()$ (for details use $?col/?row$)

```r
# (a) using "for" loops
double_sum <- function(n) {
  mid_sum <- 0
  for (r in 1:n){
    for (c in 1:r){ # inner sum only goes up to r
      mid_sum <- mid_sum + (c^2)/(5+10*r^3)
    }
  }
  return(mid_sum)
}

# example: n = 10
double_sum(10)
```

```
## [1] 0.4668784
```

(b) Idea: think of an $n \times n$ matrix where r is the row number and c the column number. Because c only goes up to r, we only need the lower triangle including the diagonal. For $n = 3$:

$$R = \begin{pmatrix}1 & 1 & 1\\ 2 & 2 & 2\\ 3 & 3 & 3\end{pmatrix}, \quad C = \begin{pmatrix}1 & 2 & 3\\ 1 & 2 & 3\\ 1 & 2 & 3\end{pmatrix}, \quad C \le R : \begin{pmatrix}T & F & F\\ T & T & F\\ T & T & T\end{pmatrix}.$$

We compute $\frac{c^2}{5+10r^3}$ for the whole matrix, keep only the entries where $c \le r$ and sum them up.

```r
double_sum_b <- function(n) {
  M <- matrix(0, n, n)
  R <- row(M)
  C <- col(M)
  relevant_entries <- C <= R
```

```
  sum((C^2 / (5 + 10 * R^3))[relevant_entries])

}

# example: n = 10
double_sum_b(10)
```

```
## [1] 0.4668784
```

Exercise 41 – Coin Change Problem

How many different ways can we make change of a USD 1 dollar bill, given half-dollars, quarters, dimes, nickels and pennies? (I.e., coins in denominations of $50, 25, 10, 5$ and 1 cents.) Implement a function which computes the number of ways to change a given amount a using k kinds of coins with denominations $d_1, ..., d_k$ using the observation that this number is the sum of the number of ways to change a without using the first kind of coin and the number of ways to change $a - d$ using all kinds of coins. (Convince yourself to count one way if the amount is 0, but zero ways if the amount is negative or there are no kinds of coins left.)

How many different ways can we make change of EUR 1 using Euro coins?

Let $W(a; d_1, \dots, d_k)$ be the number of ways to change a cents. Either we don't use the first coin at all, or we use it at least once (then d_1 is subtracted, but the coin stays available):

$$W(a; d_1, \dots, d_k) = W(a; d_2, \dots, d_k) + W(a - d_1; d_1, \dots, d_k),$$

with $W(0; \dots) = 1$, and $W(a; \dots) = 0$ if $a < 0$ or no coins are left.

```
# amount = what we want to change (in cents), coins = coins we can use
ways_to_change <- function(amount, coins) {
  if (amount == 0) { # nothing left to change -> exactly one way
    return(1)
  }
  if (amount < 0 || length(coins) == 0) { # overshot or no coins left
    return(0)
  }
  # ways without the first coin
  without_first <- ways_to_change(amount, coins[-1])

  # ways using the first coin at least once:
  # subtract it from the amount, but keep it available
  with_first <- ways_to_change(amount - coins[1], coins)

  return(without_first + with_first)
}

# USD
ways_to_change(100, c(50, 25, 10, 5, 1))
```

```
## [1] 292
```

```
# EUR
ways_to_change(100, c(50, 20, 10, 5, 2, 1))
```

```
## [1] 4562
```

So there are 292 ways for USD and 4562 for EUR. If we also count the 1 euro coin itself (the 2 euro coin can't be used for 1 euro anyway), we get exactly one more way. We left it out above since swapping 1 euro for 1 euro isn't really change:

```
ways_to_change(100, c(200, 100, 50, 20, 10, 5, 2, 1))
```

```
## [1] 4563
```

We use cents as whole numbers. With decimals (like 0.1), rounding errors could break the `amount == 0` check.

Exercise 42 – Modular Exponentiation

The remainder of the (integer) division of x by m is also called (the remainder of) x modulo m. Verify that the remainder of x times y modulo m can be obtained as the remainder of the product of the remainders of x and y modulo m modulo m. Use this to implement a recursive function *expmod* which computes the remainder of a^x modulo m without exponentation. (Distinguish the cases where x is zero, even or odd.)

We want to check that

$$(x \cdot y) \bmod m = \big((x \bmod m)(y \bmod m)\big) \bmod m.$$

```
verification <- function(x,y,m) {
  if (x%%1!=0 || y%%1!=0 || m%%1!=0) {
    return("Please only insert integers")
  }
  if (m <= 0 ) {
    return("Modulo base m must be positve")
  }
  if ((x*y)%%m == ((x%%m)*(y%%m))%%m) {
    return("It can be verified")
  }
  else {
    return("It can't be verified")
  }
}

# examples: (x,y,m) = (1,2,3), (7,9,2), (632,190,15)
verification(1,2,3)
```

```
## [1] "It can be verified"
```

```
verification(7,9,2)
```

```
## [1] "It can be verified"
```

```
verification(632,190,15)
```

```
## [1] "It can be verified"
```

For `expmod` we use $a^x = \left(a^{x/2}\right)^2$ for even x and $a^x = a \cdot a^{x-1}$ for odd x. Together with the rule above this gives

$$a^x \bmod m = \begin{cases} 1 \bmod m, & x = 0, \\ \left(\left(a^{x/2} \bmod m\right)^2\right) \bmod m, & x \text{ even}, \\ \left(a \cdot \left(a^{x-1} \bmod m\right)\right) \bmod m, & x \text{ odd}. \end{cases}$$

The recursion stops at $x = 0$, and since we take mod m at every step the numbers never get large.

```
expmod <- function(a,x,m) {
  if (x==0) {
    return(1%%m)
  }
  if (x%%2 == 0) { # x even
    t <- expmod(a, x/2, m)
    return((t*t)%%m)
  }
  if (x%%2 == 1) { # x odd, x-1 is even so the next call uses the even case
    return((a*expmod(a, x-1, m))%%m)
  }
}

# examples: (a,x,m) = (3,5,4), (10,7,3), (4,15,7)
expmod(3,5,4)
```

```
## [1] 3
```

```
expmod(10,7,3)
```

```
## [1] 1
```

```
expmod(4,15,7)
```

```
## [1] 1
```

Exercise 43 – 32-bit Biased Integers

What are the smallest and largest 32-bit integers representable by the biased scheme?

In the biased scheme a stored bit pattern $s \in \{0, \ldots, 2^{32} - 1\}$ represents the value $s - \text{bias}$ with $\text{bias} = 2^{31} - 1$. So

$$\text{smallest} = 0 - (2^{31} - 1) = -2^{31} + 1, \qquad \text{largest} = (2^{32} - 1) - (2^{31} - 1) = 2^{31}.$$

```
n_bits <- 32
bias <- 2^(n_bits-1) - 1

# as.integer because 2^n is stored as a double in R
smallest <- 0 - as.integer(bias)
smallest
```

```
## [1] -2147483647
```

```r
largest <- (2^n_bits - 1) - as.integer(bias)
largest
```

```
## [1] 2147483648
```

Exercise 44 – Two's Complement Interpretation

Using the two's complement representation for signed integers, what are the numbers corresponding to the k-bit sequences $1\cdots1$ and $10\cdots01$, respectively? (See http://en.wikipedia.org/wiki/Two%27s_complement.)

In two's complement the first bit counts as -2^{k-1} and all other bits as usual. So, using the geometric series,

$$1\cdots1 \;=\; -2^{k-1} + \sum_{j=0}^{k-2} 2^j \;=\; -2^{k-1} + \left(2^{k-1} - 1\right) \;=\; -1,$$

$$10\cdots01 \;=\; -2^{k-1} + 2^0 \;=\; -2^{k-1} + 1.$$

For example with $k = 8$ that's -1 and -127:

```r
k <- 8
val_1s <- -1
val_10_01 <- -2^(k-1) + 1
val_1s
```

```
## [1] -1
```

```r
val_10_01
```

```
## [1] -127
```

Exercise 45 – Two's Complement Conversion

Write a function which computes the k-bit (default: $k = 32$) two's complement representation of a given integer. Also implement the inverse function.

A negative x (with $-2^{k-1} \le x < 0$) has the same k bits as the positive number $2^k + x$, so we can just convert that to binary. For the inverse, if $b_1, \ldots, b_k$ are the bits,

$$x = \sum_{j=1}^{k} b_j \, 2^{k-j} \;-\; b_1 \, 2^k,$$

i.e. the normal binary value (as in exercise 35), minus 2^k if the first bit is 1.

```r
int_to_twos_comp <- function(n, k = 32) {
  # Mathematical limit check:
  # A k-bit two's complement integer can only represent values
  # strictly within the bounds of [-2^(k-1), 2^(k-1) - 1].
  lower_bound <- -(2^(k - 1))
  upper_bound <- (2^(k - 1)) - 1

  if (n < lower_bound || n > upper_bound) {
```

```r
    stop("Mathematical error: Number is outside the representable range for k bits.")
  }

  # For negative numbers, Two's complement representation is mathematically
  # identical to the standard unsigned binary representation of (2^k + n)
  if (n < 0) {
    val_to_convert <- (2^k) + n
  } else {
    val_to_convert <- n
  }

  # Convert the selected positive value to binary
  bin_vector <- numeric(0)

  if (val_to_convert == 0) {
    bin_vector <- c(0)
  } else {
    while (val_to_convert > 0) {
      remainder <- val_to_convert %% 2
      bin_vector <- c(remainder, bin_vector)
      val_to_convert <- val_to_convert %/% 2
    }
  }

  # Pad with leading zeros to ensure the vector has exactly k bits
  padding_length <- k - length(bin_vector)
  if (padding_length > 0) {
    bin_vector <- c(rep(0, padding_length), bin_vector)
  }

  return(bin_vector)
}


twos_comp_to_int <- function(b) {
  # Mathematical limit check: Vector must contain strictly 0s and 1s.
  if (any(!b %in% c(0, 1))) {
    stop("Mathematical error: Binary vector must only contain 0s and 1s.")
  }

  k <- length(b)
  if (k == 0) stop("Mathematical error: Binary sequence cannot be empty.")

  # Generate powers of 2 for positional conversion (from k-1 down to 0)
  powers <- (k - 1):0

  # Calculate the standard unsigned decimal value
  unsigned_val <- sum(b * (2^powers))

  # Evaluate the Most Significant Bit (the first element in the vector).
  # If it is 1, the number is negative, and its true value is the
  # unsigned representation minus 2^k.
  if (b[1] == 1) {
```

```
    return(unsigned_val - (2^k))
  } else {
    return(unsigned_val)
  }
}
# examples: bits = 0 0 0 0 0 1 0 1 and 1 1 1 1 1 0 1 1
int_to_twos_comp(23)
```

```
##  [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1
```

```
twos_comp_to_int(c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0
```

```
## [1] 23
```

```
twos_comp_to_int(c(1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
1,1,1,1,1,1,1,0,0,1,1,0,0,0,1))
```

```
## [1] -207
```

Exercise 46 – Rational Numbers

Rational numbers are fractions of the form p/q with integers p and $q > 0$ satisfying $gcd(p, q) = 1$. We can represent such numbers as sequences $c(p, q)$ of length two. Write a function make_rat which for two given numbers p and q returns the corresponding rational number as a sequence of length 2 (e.g., make_rat(19, −95) $\Rightarrow c\ (-1, 5)$.) Also write functions numer and denom for extracting the numerator and denominator of such a rational number.

```
# uses gcd from exercise 26
make_rat <- function(p,q) {
  if (q == 0) {
    return("Denominator q cannot be zero")
  }
  if (q < 0) { # denominator should be positive, so flip both signs
    p <- -p
    q <- -q
  }
  g <- gcd(p, q) # divide both by the gcd to simplify
  p <- p / g
  q <- q / g
  c(p, q)
}

# examples: (p,q) = (10,30), (19,-95), (1812, -902344)
make_rat(10,30)
```

```
## [1] 1 3
```

```
make_rat(19,-95)
```

```
## [1] -1  5
```

```
make_rat(1812,-902344)
```

```
## [1]   -453 225586
```

```
# numerator of the simplified fraction
numer <- function(p, q) {
  make_rat(p, q)[1]
}

# examples: (1,3), (900,-600)
numer(1,3)
```

```
## [1] 1
```

```
numer(900,-600)
```

```
## [1] -3
```

```
# denominator of the simplified fraction
denom <- function(p, q) {
  make_rat(p, q)[2]
}

# examples: (1,3), (900,-600)
denom(1,3)
```

```
## [1] 3
```

```
denom(900,-600)
```

```
## [1] 2
```

Exercise 47 – Rational Addition and Multiplication

Write functions add_rat and mul_rat for the rational addition and multiplication of rational numbers.

We use

$$\frac{p_1}{q_1} + \frac{p_2}{q_2} = \frac{p_1 q_2 + p_2 q_1}{q_1 q_2}, \qquad \frac{p_1}{q_1} \cdot \frac{p_2}{q_2} = \frac{p_1 p_2}{q_1 q_2},$$

and then simplify the result with `make_rat`.

```
#--- Add_rat
add_rat <- function(p_1, q_1, p_2, q_2){
if (q_1 == 0 || q_2 == 0){

return("This is undefined. Please check that the dnominator is not 0")
}
numerator <-numer(p_1,q_1)*denom(p_2,q_2) +numer(p_2,q_2)* denom(p_1,q_1)
denominator <-denom(p_1,q_1)*denom(p_2,q_2)
return(make_rat(numerator,denominator))
}
add_rat(2,4,2,5)
```

```
## [1]  9 10
```

```
#---Mul_rat
mul_rat<-function(p_1,q_1,p_2, q_2){
if (q_1== 0|| q_2 == 0){
return("This is undefined.Please check that the dnominator is not 0")
}
x<-make_rat(p_1,q_1)
y<-make_rat(p_2,q_2)
multiplied<-x* y
mul_rational<-make_rat(multiplied[1],multiplied[2])
return(mul_rational)
}
mul_rat(2,4,2,5)
```

```
## [1] 1 5
```